

# PDF Parsing Library Comparison

## Task 2 — Resume Parsing Project

For this task I looked at three Python libraries that can be used to pull text out of PDF resumes — **PyMuPDF**, **pdfplumber**, and **pdfminer.six**. Instead of just reading about them, I actually installed all three and ran them on the same real resume PDF so I could see for myself how they perform in terms of installing, speed, accuracy, and whether they keep the resume's formatting/order intact.

| Criteria                          | PyMuPDF (fitz)                                                                                                                          | pdfplumber                                                                                     | pdfminer.six                                                                                           |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <b>Installation</b>               | pip install pymupdf.<br>Initially failed with a DLL error; fixed by installing the Microsoft Visual C++ Redistributable and restarting. | pip install pdfplumber.<br>Installed and imported without issues.                              | pip install pdfminer.six.<br>Installed and imported without issues.                                    |
| <b>Ease of use</b>                | Simple API (pymupdf.open, get_text()). Note: the 'fitz' import name is now deprecated in favor of 'import pymupdf'.                     | Very simple API (pdfplumber.open, extract_text()).                                             | Simple function call (extract_text()), fewer config options.                                           |
| <b>Speed</b>                      | Fastest of the three — returned results almost instantly (per documentation and observed behavior).                                     | Moderate — slightly slower than PyMuPDF but fast enough for resumes.                           | Generally the slowest of the three on larger files.                                                    |
| <b>Accuracy</b>                   | High — all text captured, but moved the header text ('Resume: Digital Arts') from the top to the end of the page.                       | High — text extracted correctly, no missing characters, order matched the original exactly.    | High, but added stray blank lines and one garbled symbol not present in the original PDF.              |
| <b>Formatting / reading order</b> | Good, but split job titles and their dates onto separate lines instead of keeping them together.                                        | Best — job titles and dates stayed on the same line, matching the original layout exactly.     | Weaker — grouped all job titles first, then all dates separately, breaking the original reading order. |
| <b>Advantages</b>                 | Very fast; handles images and complex PDFs well (per documentation).                                                                    | Best layout preservation; also extracts tables, useful for skills/education tables in resumes. | Mature, widely used; good fallback option.                                                             |
| <b>Limitations</b>                | Needs the Visual C++ Redistributable installed to work on Windows — an extra setup step beyond pip install.                             | Slightly slower than PyMuPDF on very large files (not an issue for single resumes).            | Reading order can get scrambled on multi-column or visually complex resumes.                           |

## Test Method

I installed all three libraries in the same Python setup, then ran each one on page 3 of a sample resume PDF I downloaded from a university career center (it had 17 example resumes in it, so I picked one — a Digital Arts resume — to test on). Then I just compared what each library printed out against how the resume actually looks, to see if the text came out in the right order and nothing important got lost.

## Key Findings

### *1. PyMuPDF (pymupdf/fitz) — fast, but gave me some setup trouble*

This one was installed fine with pip, but when I tried to actually import it, I got a DLL error. Turns out I needed to install the Microsoft Visual C++ Redistributable and restart my PC before it would work — so it's not as plug-and-play as the other two. Once I got it running though, the text extraction was accurate, but it moved the header text at the top of the page ('Resume: Digital Arts / Font: Verdana') to the very end of the output instead of keeping it at the top, and it put job titles and their dates on separate lines instead of together.

### *2. pdfplumber — this one gave the best results*

Out of the three, pdfplumber's output looked the closest to the actual resume. Job titles and their dates stayed on the same line like in the original, and section headings like EDUCATION, PROFESSIONAL EXPERIENCE, and SKILLS were easy to spot. I think this will make the section-detection task (Task 5) a lot easier later on, since the layout stays predictable.

### *3. pdfminer.six — got the words, but the order got messed up*

pdfminer.six pulled out basically the same words as the other two, but it grouped all the job titles together first and then dumped all the dates in a separate block afterward, instead of keeping them lined up like in the actual resume. There were also a bunch of extra blank lines, and one weird symbol showed up near the end of the output that isn't anywhere in the real PDF. It still technically works, but I'd need to clean it up a lot more than the pdfplumber output.

## Recommendation

Based on what I saw, I'm going with **pdfplumber** for the text extraction module in Task 3. It gave the cleanest results — the reading order matched the original resume the closest, and it also has a built-in way to pull out tables, which could come in handy if a resume lists skills or education in a table. PyMuPDF was noticeably faster and is still a good option, but needing the Visual C++ Redistributable installed first felt like an extra hurdle, and it shuffled some content around on my test page. pdfminer.six worked too, but I'd only really use it as a backup since its output needs more cleanup.

## Appendix: Sample Extraction Output

### PyMuPDF output (excerpt):

```
Graphic Designer
August 2019 - Present
Freelance Web & Graphic Design
...
SKILLS
```

...

Resume: Digital Arts

Font: Verdana

(header text moved from top of page to the end of the output)

#### **pdfplumber output (excerpt):**

Allie Gator Gainesville, FL 32611 | 352-290-5678 | albertagator@ufl.edu

EDUCATION

Bachelor of Arts in Digital Arts June 2023

PROFESSIONAL EXPERIENCE

Graphic Designer August 2019 – Present

(title and date correctly stayed on one line)

#### **pdfminer.six output (excerpt):**

Resume: Digital Arts

Allie Gator

...

Graphic Designer

Freelance Web & Graphic Design

Studio Co-Founder & Leader

Design Awaken Arts, Gainesville, FL

June 2023

August 2019 – Present

(titles grouped first, dates grouped separately afterward – reading order broken)